Value Class Emotions

Rémi Forax

Université Gustave Eiffel, May 2026

Warning, I'm not using a released JDK

This is experimental!

```
import module java.base;
IO.println(Runtime.version());
```

```
27-internal-adhoc.forax.valhalla
```

What is a value class?

Instances/references/objects are values not pointers

```
value record Point(int x, int y) {}
```

```
Point p1 = new Point(1, 2);
Point p2 = new Point(1, 2);
IO.println(p1 == p2);
```

```
true
```

A value class has **no identity**, no address in memory

The operator == compares all the field values

Why OpenJDK Valhalla ?

- No cost abstraction
- Flat representation (CPU friendly)
- Primitives are a nuisance

Value type objects are:

- **unmodifiable**,
- **identity-free**,
- stored and passed **by value** rather than by pointer

No overhead of heap allocation and pointer indirection for small data structures

Is it like a struct in C?

No, a value class is unmodifiable!

A value class is **implicitly `final`**

All fields are **implicitly strict `final`**

```
value class MyInteger {
  int x;
}
```

```
Error: strict field x is not initialized before the supertype constructor has been called
```

Are value instances objects?

Yes !

It can also implement interfaces and extend an abstract class

```
value record Person(String name) implements Comparable<Person> {
  @Override
  public int compareTo(Person o) {
    return name.compareTo(o.name);
  }
}
```

```
var person = new Person("Bob the welder");
Object object = person;   // the VM may box the value
```

toString() and hashCode() works!

```
value class Pet {
  String kind;
  public Pet(String kind) { this.kind = kind; }
}
```

```
var garfield = new Pet("cat");
var charly = new Pet("cat");
IO.println(garfield);
IO.println(Integer.toHexString(charly.hashCode()));
```

```
REPL.$JShell$11$Pet@3d4beac6
3d4beac6
```

Uses the values of the fields to compute the 'default' hashCode()

Synchronized and other methods that need a header?

An **identity** class has a header

A value class has no a header

```
value record MyFloat(float f) {}
var myFloat = new MyFloat(3.14f);
synchronized (myFloat) { }
```

```
Error: unexpected type
  required: a type with identity
  found:    MyFloat
```

```
Object o = myFloat;
synchronized (o) { }
```

```
java.lang.IdentityException: Cannot synchronize on an instance of value class REPL.$JShell$16$MyFloat
```

Three problems to solve

This is not that hard...

1. `this` in a constructor is modifiable
2. Same bytecode for identity/value class
3. Java classes are loaded lazily (very late)
   - after the type of fields is discovered
   - after the type of parameters is discovered

Strict initialization is required!

All fields of a value class must be initialized **before** the call to `super()`

```
value class MyInteger {
  int value;
  MyInteger(int value) {
    super();                // <- oops
    this.value = value;
  }
  int value() { return value; }
}
```

```
Error: strict field value is not initialized before the supertype constructor has been called
```

Strict initialization available in Java 25!

To prepare the introduction of value types

Java 25 supports strict initialization, avoids **leaky** `this`

```
class Person {
  String name;            // final or not
  Person(String name) {
    Objects.requireNonNull(name);
    this.name = name;
    super();
  }
  public String toString() { return name; }
}
new Person("John")
```

Mandelbrot set

Mandelbrot set

The set of complex numbers c for which the sequence defined by the iteration:

$z_0 = 0$ or $z_{n+1} = z_n^2 + c$

remains bounded (never escapes to infinity)

```
value record Complex(double re, double im) {
  // add(), square() ...
}
static int iterate(Complex c) {
  var z = new Complex(0, 0);
  for (var i = 0; i < MAX_ITER; i++) {
    if (z.absSquared() > 4.0) return i;  // escaped
    z = z.square().add(c);
  }
  return MAX_ITER;
}
```

Version using primitives

This version is less readable

```
static int iterate(double cx, double cy) {
  var zx = 0.0;
  var zy = 0.0;
  for (var i = 0; i < MAX_ITER; i++) {
    var zx2 = zx * zx;
    var zy2 = zy * zy;
    if (zx2 + zy2 > 4.0) return i;   // escaped
    zy = 2 * zx * zy + cy;
    zx = zx2 - zy2 + cx;
  }
  return MAX_ITER;
}
```

Benchmarks

1024 x 1024 — itérations max : 255

```
Benchmark    | Mode | Cnt | Score     Error Units
-------------|------|-----|---------------------
primitive    | avgt | 10  | 19.758  ± 0.119 ms/op
record       | avgt | 10  | 207.828 ± 1.938 ms/op
value record | avgt | 10  | ??
```

GC usages

```
TODO
```

Value class is a VM optimization

The Java compiler does almost nothing

The JIT (c2) remove allocation/indirection when the bytecode is transformed to machine code

Existing JDK classes retrofitted as value classes

All existing classes annotated with `@ValueBased` are now value classes

All wrappers `java.lang.Boolean`, `java.lang.Integer`, etc

`java.util.Optional`

Most classes of `java.time`

```
IO.println(Boolean.class.isValue());
```

```
true
```

Storing value instances in fields/arrays?

Works but may not get the best performance

```
value record Person(String name, int age) {}
class Car {
  Person driver;
  int numberOfSeats;
}
```

Reading/Storing the value instance `driver` in RAM may require **several read/writes**

The VM spec mandates reference read/write to be "atomic"

So only 64 bits value instances (`null` included) are flattened?

JEP 401: Value Classes and Objects (Preview)

Mantra: Code like a class, Work like an int

Scalarization is done by the JIT

Flattening if size <= 64 bits (`null` included)

No need to **recompile** the user code (not fully true)

Retrofit `Integer`, `Optional`, `LocalDate`, etc to be value classes

Value Class Emotions

Let's try to imrpove the heap flattening

Idea of the Value Class Emotions

Let's help flattening by adding type emotions

'!' or '?' sigils at the end of a type to indicate **nullability**

`Boolean!`, `Optional!`, `Complex?`, etc.

The emotion bang '!'

Extends Java to add '!' at the end of a type of a field

```
value record Person(String name, int age) {}
class Car {
  Person! driver;
}
```

```
Error: null restricted field driver is not initialized before the supertype constructor has been called
```

Fields with '!' has to be initialized before super()

A null-restricted field can **not be set** to 'null'

```
value record Person(String name, int age) {}
class Car {
  Person! driver;
  Car(Person driver) {
    this.driver = driver;   // the VM throws a NPE
    super();
  }
}
```

```
//new Car(null);
```

Method parameters with '!'

Equivalent to `Objects.requireNonNull()` on the parameter

```
value record Person(String name, int age) {}
class Car {
  // Person! driver;
  Car(Person! driver) {
    // this.driver = driver;
    super();
  }
}
```

```
new Car(null);
```

```
java.lang.NullPointerException: null
```

Creating an array with '!'

Same issue, the array elements **can not be initialized** to `null`

Without initial elements

```
var array = new Person![4];
```

```
Error: Non-empty null-restricted array of type Person! must specify an initializer
```

Use an existing array as prototype

```
var proto = new Person[4];
Arrays.setAll(proto, _ -> new Person("Bob", 42));
var array = (Person[]) Array.newInstance(Person.class, 0x0200, 4, proto, 0);
```

Using an array with '!'

As with fields, the VM checks at runtime

```
var proto = new Person[4];
Arrays.setAll(proto, _ -> new Person("Bob", 42));
var array = (Person[]) Array.newInstance(Person.class, 0x0200, 4, proto, 0);
```

```
array[1] = null;
```

```
java.lang.NullPointerException: Cannot store null in a null-restricted array
```

```
Object[] objectArray = array;
objectArray[1] = null;
```

```
java.lang.NullPointerException: Cannot store null in a null-restricted array
```

Why not using '?' instead of '!'?

Like in Kotlin?

```
String s = null;   // Invalid in Kotlin, valid in Java 26
```

This is **not a backward compatible** change

The semantics of '!' is weird?

We want to be backward compatible, so '!' can not be used in method selection

```
class A { void m(Object o) {} }
class B extends A { void m(Complex! c) {} }
```

```
B b = new B();
b.m(null);
```

```
java.lang.NullPointerException: null
```

Flattening with bangs

```
value record Complex(double re, double im) {}
class Maybe {
  Boolean! flag;
  final Complex! value;
  Maybe(Boolean! flag, Complex! value) {
    this.flag = flag;
    this.value = value;
    super();
  }
}
```

`flag` is flatten (sizeof <= 64 bits), `value` is flatten (`final` bang)

Summary

Code like a class, Work like an int

value type instances are scalarized on stack

```
var c = new Complex(0, 0);
```

Value type fields are flattened on heap

- if sizeof <= 64 bits (1 byte for `null`)
- if sizeof <= 64 bits and '!'
- if the field is `final` and '!'

```
record Line(Complex! c1, Complex! c2) { }
```

Roadmap to Valhalla

🚚JEP 513: Flexible Constructor Bodies

🏗️JEP 401: Value Classes and Objects (Preview)

🚧JEP Draft: Null-Restricted Value Class Types (Preview)

☁️JEP 402: Enhanced Primitive Boxing (int ≈ Integer!)

🚧Type Classes (operator overloading)

☁️Parametric JVM (List<ValueType>)